Predictive: statistical models

Vittorio Maniezzo
Univ. Bologna

Forecast, selected algorithms

Time series forecasting classical methods:

- Autoregression (AR)
- Vector Autoregression (VAR)
- Moving Average (MA)
- Autoregressive Moving Average (ARMA)
- Vector Autoregression Moving-Average (VARMA)
- Vector Autoregression Moving-Average with Exogenous Regressors (VARMAX)
- Autoregressive Integrated Moving Average (ARIMA)
- Seasonal Autoregressive Integrated Moving-Average (SARIMA)
- Seasonal Autoregressive Integrated Moving-Average with Exogenous Regressors (SARIMAX)
- Simple Exponential Smoothing (SES)
- Holt and Winter's Exponential Smoothing (HWES)
- BATS (and TBATS)

Autoregressive models AR(p)

In autoregressive models (of order p, AR(p)) the future value of a variable (added to a random component and to a constant) is assumed to be a linear combination of the last p observations

$$y_t = c + \sum_{i=1}^{p} \varphi_i y_{t-i} + \varepsilon_t$$

Feasible only for stationary processes! It only models a linear dependency.

where:

y_t value at time t

ε_t random error at time t

φ_i $(i = 1, \dots, p)$ model parameters

c constant

Fil rouge, AR(2)

On log-diff data to ensure stationarity.
Following LMS analysis on data lagged up to p periods.

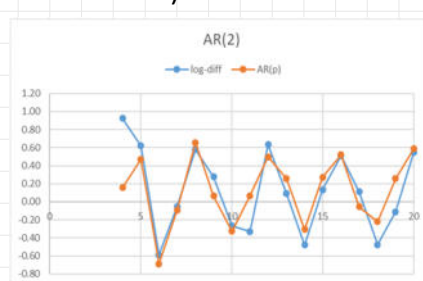
t	y_t	log-diff	t-1	t-2	AR(p)
1	128				
2	181	0.35			
3	87	-0.73	0.35		
4	219	0.92	-0.73	0.35	0.16
5	407	0.62	0.92	-0.73	0.47
6	226	-0.59	0.62	0.92	-0.69
7	214	-0.05	-0.59	0.62	-0.09
8	383	0.58	-0.05	-0.59	0.66
9	505	0.28	0.58	-0.05	0.06
10	387	-0.27	0.28	0.58	-0.33
11	278	-0.33	-0.27	0.28	0.07
12	523	0.63	-0.33	-0.27	0.50
13	572	0.09	0.63	-0.33	0.26
14	354	-0.48	0.09	0.63	-0.31
15	404	0.13	-0.48	0.09	0.27
16	673	0.51	0.13	-0.48	0.52
17	752	0.11	0.51	0.13	-0.06
18	468	-0.47	0.11	0.51	-0.22
19	419	-0.11	-0.47	0.11	0.26
20	725	0.55	-0.11	-0.47	0.59

Model $y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2}$

with

c	0.197
φ_1	-0.301
φ_2	-0.754

(regression, y↔log-diff, x ↔ t-1 e t-2)



Autoregression (AR)

AR models the next step in the sequence as a linear function of the observations at prior time steps.

The notation for the model specifies the order of the model p as a parameter to the AR function, e.g. AR(p).

The method is suitable for univariate time series without trend and seasonal components.

```
from statsmodels.tsa.ar_model import AutoReg
# artificial dataset
data = [x + np.random.random() for x in range(1, 100)]
# fit model
model = AutoReg(data, lags=1)
model_fit = model.fit()
# make prediction (from-to)
pred = model_fit.predict(0, len(data))
fore = model_fit.forecast(4)
print(f"[AR]: {fore}")
```


Akaike Information Criterion

AR are a first example of model of a time series. But *how good is the model*?

The Akaike Information Criterion (AIC) is an *estimator of the prediction error*, i.e., of the relative quality, of statistical models *for a given set of data*. It estimates how much information is lost by using the model instead of the actual data.

Suppose that we have a model of some data. Let k be the number of estimated parameters in the model, the AIC value of the model is:

$$AIC = -2\log L + 2k$$

where k is the number of estimated parameters and L is the maximum likelihood of the model.

Given a set of candidate models for the data, the preferred model is the one with the minimum AIC value.

Model likelihood

The likelihood expression can be complicated, but if

- the errors are normally distributed (residuals are gaussian)
- the sample size is large enough

The term $\log L$ (log-likelihood) can be approximated by:

$$\log L = -\frac{n}{2}(\log(2\pi) + \log(\sigma^2) + 1)$$

where σ^2 is the variance of the residuals

AIC example

Suppose that there are 3 candidate AR(2) models, with AIC values AIC_1, AIC_2, AIC_3. Let AIC_{min} be their minimum.

The quantity $e^{((AIC_{min}-AIC_i)/2)}$ is proportional to the probability that the *i-th* model is the one that minimizes the information loss

For AR(2) models $k = 3$ (parameters c, ϕ_1, ϕ_2)

We have n=40 samples, variances are 10, 12, 15.

The AIC values result to be 109, 112, and 117.

- The second model is $e^{((109-112)/2)}$ = 0.161 times as probable as the first (min) one to minimize the information loss.
- The third model is $e^{((109-117)/2)}$ = 0.017 times as probable as the first (min) one to minimize the information loss.

AICc, corrected AIC

When the sample size is small, AIC will select models that have too many parameters (AIC will prefer overfits).

AICc is AIC with a correction for small sample sizes.

The formula for AICc depends upon the model. If the model is univariate, is linear in its parameters, and has normally-distributed residuals, then the formula for AICc is:

$$AICc = AIC + \frac{2k^2 + 2k}{n - k + 1}$$

where n is the sample size and k the number of parameters.

(AICc is AIC with an extra penalty term for the number of parameters)

LASSO

LASSO (L1 regularization) stands for *Least Absolute Shrinkage and Selection Operator*. It performs selection and regularization (constrains values) of the coefficients of the linear model.

It has been extended to other statistical models.

Example: given a linear regression model $y = \beta_0 + \beta_1 x$ LASSO requires to solve the problem

$$\min_{\beta_0,\beta_1} \sum_{i=1}^{n} \left(y_i - (\beta_0 + \beta_1 x_i)\right)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

(in vector format $\min_{\boldsymbol{\beta}} \|y - \boldsymbol{\beta X}\|_2^2 + \lambda \, \|\boldsymbol{\beta}\|_1$ where λ is a parameter)

LASSO constraints the absolute values of the coefficients. They can be zero, removing the corresponding variable.

Mostly used for feature selection in multivariate forecasting, it can be adapted to the univariate case.

To use lasso for univariate forecast, one must transform the time series into a supervised learning problem by creating a matrix of lagged features (see MLP)

LASSO example

```
y = pd.read_csv('Boxjenkins.csv', usecols=[1]).values.flatten()
logsales = np.log(y)
logdiff  = np.array([logsales[i] - logsales[i-1] for i in range(1,len(logsales))])
model = AutoReg(logdiff, lags=5)
model_fit = model.fit()
pred = model_fit.predict(0, len(logdiff))
plt.plot(logdiff)
plt.plot(pred)
plt.show()

# Create and fit the LASSO regression model, create lagged features for AR(12) model
p = 12  # AR order (number of lags)
X = np.column_stack([y[i:-p+i] for i in range(p)])  # Lagged features (y_{t-1}, y_{t-2}, ..., y_{t-p})
y_target = y[p:]  # Target variable (y_t)
X_train, X_test, y_train, y_test = train_test_split(X, y_target, test_size=0.2, random_state=42)
lasso = Lasso(alpha=0.1)  # alpha is the regularization strength
lasso.fit(X_train, y_train)
print("Coefficients:", lasso.coef_)
predictions = lasso.predict(X_test)

plt.plot(predictions)
plt.show()
# Plot the results
plt.scatter(logdiff, predictions, color='blue', label='Predictions')
plt.plot([logdiff.min(), logdiff.max()], [logdiff.min(), logdiff.max()], color='red', linestyle='--',
label='Ideal Fit')
plt.xlabel('Actual logdiff')
plt.ylabel('Predicted logdiff')
plt.title('LASSO Regression: Actual vs Predicted')
plt.legend()
plt.show()
```


Moving average models MA(q)

In moving average models (of order q, MA(q)) the future value of a variable is assumed to be equal to the average of the observations, μ, added to a linear combination of the last q errors:

$$y_t = \mu + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

Where:

μ average of the series

θ_j $(j = 1, \dots, q)$ model parameters

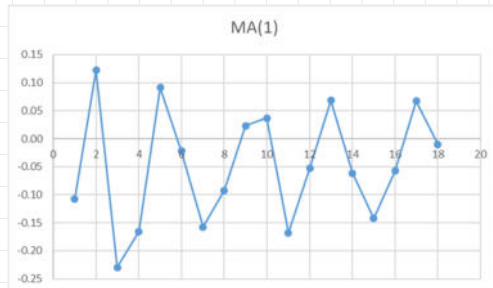
Random errors are usually assumed to have a normal distribution (mean 0, variance σ^2)

Fil rouge, MA(1)

MA is just a regression on previous periods errors

ε_t	-0.06
θ_1	-0.21

log-diff	y_t-*m*	y_{t-1}-*m*	*MA(1)*
0.35			
-0.73			-0.11
0.92	0.80		0.12
0.62	0.50	0.80	-0.23
-0.59	-0.71	0.50	-0.17
-0.05	-0.18	-0.71	0.09
0.58	0.46	-0.18	-0.02
0.28	0.15	0.46	-0.16
-0.27	-0.39	0.15	-0.09
-0.33	-0.46	-0.39	0.02
0.63	0.51	-0.46	0.04
0.09	-0.04	0.51	-0.17
-0.48	-0.60	-0.04	-0.05
0.13	0.01	-0.60	0.07
0.51	0.39	0.01	-0.06
0.11	-0.01	0.39	-0.14
-0.47	-0.60	-0.01	-0.06
-0.11	-0.24	-0.60	0.07
0.55	0.42	-0.24	-0.01



(there is a hint of seasonality in the error)

Moving Average (MA)

The notation for the model involves specifying the order of the model q as a parameter to the MA function, e.g. MA(q). For example, MA(1) is a first-order moving average model.
The method is suitable for univariate time series without trend and seasonal components.
Unavailable alone. To showcase it we must use the ARIMA class (see forward) and create an MA model setting a zeroth-order AR model.

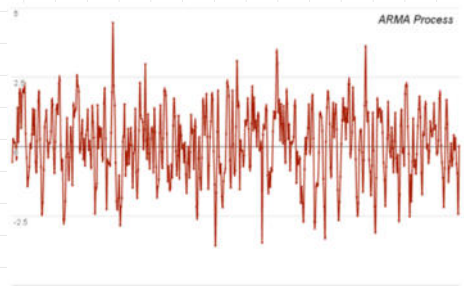
```
# MA example
from statsmodels.tsa.arima.model import ARIMA
from random import random
data = [x + random() for x in range(1, 100)] # artificial dataset
# fit model
model = ARIMA(data, order=(0, 0, 1))
model_fit = model.fit()
# make prediction
yhat = model_fit.predict(len(data), len(data))
print(yhat)
```

ARMA models

Autoregressive models (AR) and moving average models (MA) can be combined in a more general class, the ARMA models.

In mathematics, an $ARMA(p,q)$ model is:

$$y_t = c + \sum_{i=1}^{p} \varphi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$



Autoregressive Moving Average (ARMA)

ARMA models the next step in the sequence as a linear function of the observations and residual errors at preceding time steps, combining Autoregression (AR) and Moving Average (MA) models. The method is suitable for univariate time series without trend and seasonal components.
Again, we downscale ARIMA to show an ARMA(2,1).

```
data = pd.read_csv("FilRouge.csv",usecols=["sales"]).values
# fit model
model = ARIMA(data, order=(2, 0, 1))
model_fit = model.fit()
# make prediction (in-sample or out-of-sample)
pred = model_fit.predict(0, len(data))
# make forecast (out of sample)
fore = model_fit.forecast(4)
print(f"[ARMA]: {fore}")
```

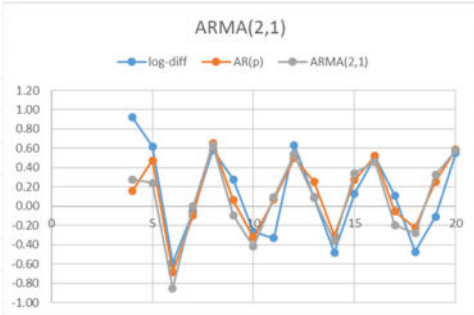
Hyperparameters, defined externally

Differences are elusive

Fil rouge, ARMA(2,1)

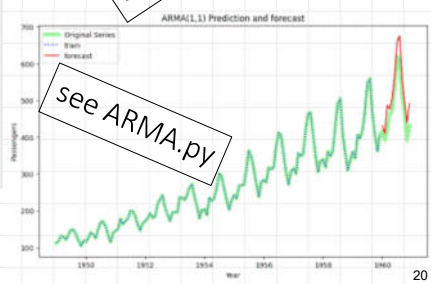
MA improves a little over AR. More data would be needed.

If the series is slightly under differenced, adding one or more additional AR terms can fix it. Likewise, if it is slightly over-differenced, adding an additional MA term helps.



see ARMAairline.ipynb

see ARMA.py



ARIMA models

ARMA models can be used only on stationary processes, without trend or seasonality.

ARIMA models (*AutoRegressive Integrated Moving Average*) are ARMA models that work on a diff time series (diff preproc), thus lift the trend application limit.

ARIMA(p,d,q) =

$$y_t = c + \sum_{i=1}^{p} \varphi_i y'_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

p order of the autoregressive component

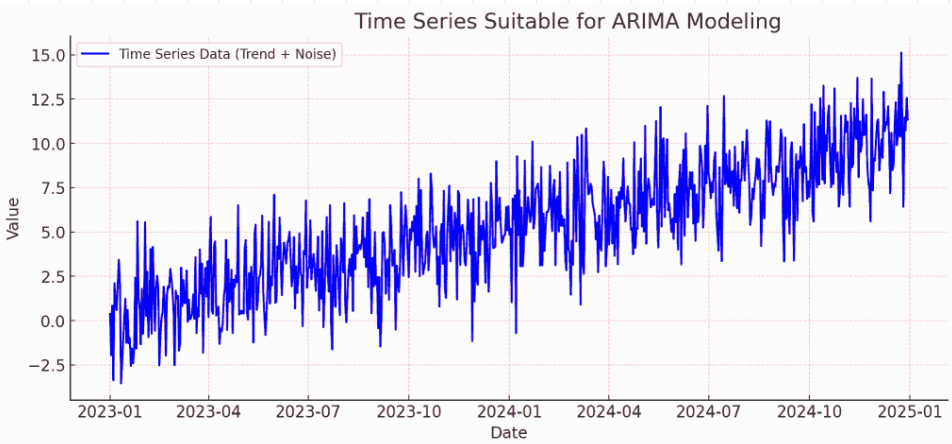
q order of the moving average component

d diff degree (usually d =1, if d=0 ARIMA=ARMA)

ARIMA model in words:

Predicted y_t = Constant
+ Linear combination lags of y (upto p lags)
+ linear combination of lagged forecast errors (upto q lags)

ARIMA process, example



ARIMA

The key aspects of the model are:

- AR: Autoregression. The model uses a relationship between an observation and some number of lagged observations.
- I: Integrated. The model uses differencing of raw observations (e.g. subtracting an observation from an observation at the previous time step) in order to make the time series stationary.
- MA: Moving Average. The model uses the dependency between an observation and a residual error from a moving average model applied to lagged observations.

ARIMA, parameters

Each of its components is explicitly specified in the model as a parameter $\mathrm{ARIMA}(p,d,q)$, defined as follows:

- p: The number of lag observations included in the model, also called the lag order.
- d: The number of times that the raw observations are differenced, also called the degree of differencing.
- q: The size of the moving average window, also called the order of moving average.

Perameters setting: grid search

Algorithms are very sensitive to their control parameters (aka hyperparameters).

Optimizing parameter setting is an active area of research, many method, from simple to very complicated.

The simplest one: grid search: define an interval for each parameter and test all combinations (integer parameters).

Example: ARIMA, parameters p,d,q, intervals: $p \in [0,3], d \in [0,2]$, $q \in [0,2]$

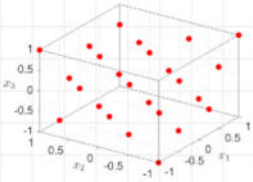
Grid search:

```
for (p=0;p<=3;p++)
   for(d=0;d<=2;d++)
      for(q=0;q<=2;q++)
      {  performance = ARIMA(p,d,q);
         if (performance > best_performance) … etc
```

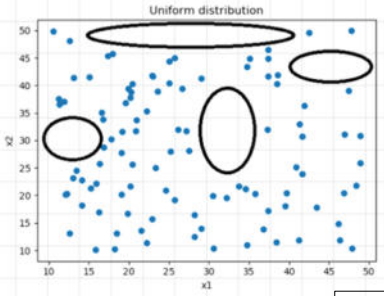
Perameters setting: Hammersley

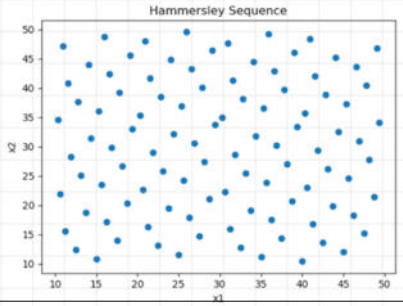
For *continuous parameters*, it is possible to randomly generate values inside the interval

Independent generation along each dimension could lead to uneven sampling in multidimensional spaces



A possible solution is Hammersley sampling





in case, see Hammersley.py, source code

Configuring an ARIMA Model

The classical approach for fitting parameters of an ARIMA model is the Box-Jenkins methodology, steps:

1. Model Identification. Identify trends, seasonality, and autoregression elements to get an idea of the amount of differencing and the size of the lag that will be required.
2. Parameter Estimation. Use a fitting procedure to find the coefficients of the regression model.
3. Model Checking. Use plots and statistical tests of the residual errors to determine the amount and type of temporal structure not captured by the model.

The process is repeated until a desirable level of fit is achieved on the in-sample or out-of-sample observations (e.g. training or test datasets).

The process was first described in George Box, Gwilym Jenkins "Time Series Analysis: Forecasting and Control", 1970.

ARIMA, utilities, data series

May be needed, may be not. Check your case. Here data as series!

```
# Load the dataset
df = pd.read_csv("M3C_monthly.csv")
rawdata = df.iloc[490,6:].values.astype(float)
train,test = rawdata[:-12], rawdata[-12:]

logdata = np.log(train)             # log transform
logdiff = pd.Series(logdata).diff() # logdiff transform

# forecast, you can do better
fore = pd.Series(0, index=range(132, 132 + 12))

# Postprocessing, reconstruction (here very pythonic, otherwise plain loops)
logdiff.iloc[0] = logdata.iloc[0]                  # set first series entry
reclogdata = pd.concat([logdiff,fore]).cumsum()
recdata = np.exp(reclogdata)

plt.plot(rawdata[:-12], label="train",linewidth=5)
plt.plot(recdata[:-12],label="prediction")
plt.plot(range(len(rawdata)-12,len(rawdata)),recdata[-12:],label="forecast")
plt.plot(range(len(rawdata)-12,len(rawdata)),rawdata[-12:], label="test")
plt.title("M3 series"),plt.xlabel("time"),plt.ylabel("value"),plt.ylim(0,10000)
plt.legend()
plt.show()
```


ARIMA, utilities, arrays

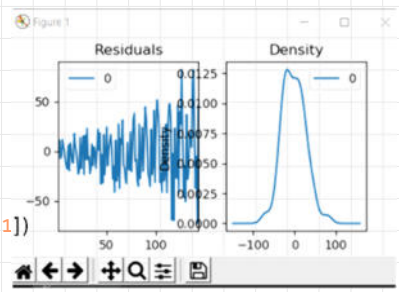
May be needed, may be not. Check your case. Here data numpy arrays!

```
# Load the dataset
df = pd.read_csv("M3C_monthly.csv")
rawdata = df.iloc[490,6:].values.astype(float)
train,test = rawdata[:-12], rawdata[-12:]

# Preprocessing, log - diff transform
logdata = np.log(train)             # log transform
logdiff = np.array([logdata[i]-logdata[i-1] for i in range(1, len(logdata))]) # logdiff transform

# forecast, you can do better
fore = np.zeros(len(test))

# Postprocessing, reconstruction
reclogdiff = np.insert(logdiff,0,logdata[0]).cumsum()
recfore = np.insert(fore,0,reclogdiff[-1]).cumsum()
recpred = np.exp(reclogdiff)
recfore = np.exp(recfore)

plt.plot(rawdata[:-12], label="train",linewidth=5)
plt.plot(recpred,label="prediction")
plt.plot(range(len(rawdata)-13,len(rawdata)),recfore,label="forecast")
plt.plot(range(len(rawdata)-12,len(rawdata)),rawdata[-12:], label="test")
plt.title("M3 series"),plt.xlabel("time"),plt.ylabel("value"),plt.ylim(0,10000)
plt.legend()
plt.show()
```

Python, ARIMA

```
import os
import pandas as pd, matplotlib.pyplot as plt
from statsmodels.tsa.arima_model import ARIMA

# Import data
df = pd.read_csv('BoxJenkins.csv', usecols=[1], names=['value'], header=0)

# 1,1,2 ARIMA Model (p,d,q)
model = ARIMA(df.value, order=(1,1,2))
model_fit = model.fit()
print(model_fit.summary())
# diff=1
pred = model_fit.predict(1, len(df), typ="levels")
fore = model_fit.forecast(4)
plt.plot(df.values)
plt.plot(pred)
plt.show()
# Plot residual errors
residuals = pd.DataFrame(model_fit.resid)
fig, ax = plt.subplots(1,2)
residuals.plot(title="Residuals", ax=ax[0])
residuals.plot(kind='kde', title='Density', ax=ax[1])
plt.show()
```

Original, non diff'ed values



SARIMA models

An extension, proposed by Box, Jenkins (1970), of ARIMA models which can be applied also to data with seasonality.

A seasonal diff is included.

Model identification can be complex.

Notationally, they are denoted by $ARIMA(p,d,q)(P,D,Q)_m$

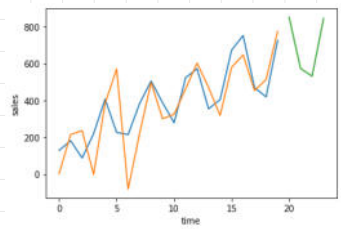
where:

- p,d,q are ARIMA parameters referring to periods (ex., week),
- P,D,Q the same parameters referring to seasons (ex., trimester)
- m number of period in one season (ex., weeks in a trimester. Multiple seasonalities will be addressed by an extension at the end of this slide pack).

Fil Rouge, SARIMA (pmdarima)

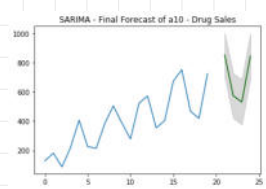
Pmdarima is just SARIMA with built-in grid search (*auto_arima*).

```
import pmdarima as pm # pip install pmdarima
import pandas as pd
df = pd.read_csv('FilRouge.csv', names=['sales'], header=0)
ds = df.sales
model = pm.auto_arima(ds.values, start_p=1, start_q=1,
                      test='adf', max_p=3, max_q=3, m=4,
                      start_P=0, seasonal=True,
                      d=None, D=1, trace=True,
                      error_action='ignore',
                      suppress_warnings=True,
                      stepwise=True) # stepwise=False full grid
print(model.summary())
morder     = model.order              # p,d,q
mseasorder = model.seasonal_order     # P,D,Q,m
fitted = model.fit(ds)
yfore  = fitted.predict(n_periods=4) # forecast
ypred  = fitted.predict_in_sample()
plt.plot(ds.values)
plt.plot(ypred.values)
plt.plot([None for i in ypred] + [x for x in yfore])
plt.xlabel('time');plt.ylabel('sales')
plt.show()
```



Fil Rouge, forecast (pmdarima)

```
data = ds.values
n_periods = 4
fitted, confint = model.predict(n_periods=n_periods, return_conf_int=True)
tindex = np.arange(df.index[-1]+1,df.index[-1]+n_periods+1)

# series, for plotting
fitted_series = pd.Series(fitted, index=tindex)
lower_series = pd.Series(confint[:, 0], index=tindex)
upper_series = pd.Series(confint[:, 1], index=tindex)

# Plot
plt.plot(data)
plt.plot(fitted_series, color='darkgreen')
plt.fill_between(lower_series.index, lower_series, upper_series,
                 color='k', alpha=.15)
plt.title("SARIMAX")
plt.show()
```



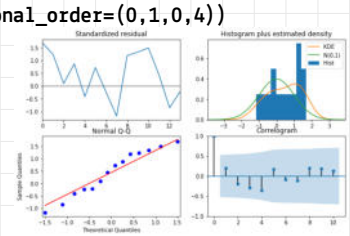
Fil Rouge, SARIMA (statsmodels)

Need to import SARIMAX (which could deal also with exogenous predictors)

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
sarima_model = SARIMAX(ds, order=(0,2,2), seasonal_order=(0,1,0,4))
sfit = sarima_model.fit()
sfit.plot_diagnostics(figsize=(10, 6))
plt.show()
```

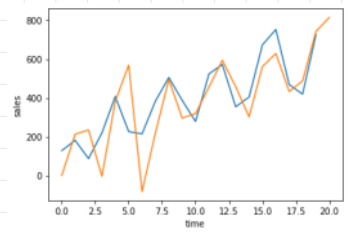
See next slide

Note: here non preprocessed data. Preprocessing: exercise



Predictions in-sample:

```
ypred = sfit.predict(start=0,end=len(df))
plt.plot(df.sales)
plt.plot(ypred)
plt.xlabel('time');plt.ylabel('sales')
plt.show()
```



Fil rouge, diagnostics

```
sfit.plot_diagnostics(figsize=(7,5))
plt.show()
```

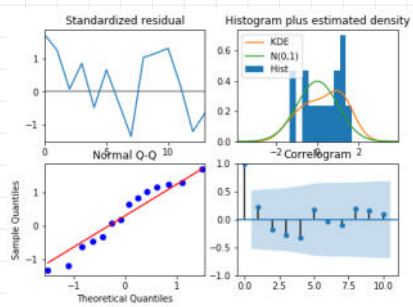
Diagnostics (*here, too feew data*!):

Top Left: Residual errors should fluctuate around the mean with uniform variance.

Top right: density plot, should be normal distribution with zero mean.

Bottom left: Q-Q plot. Points should be aligned, distant points imply skewed distribution.

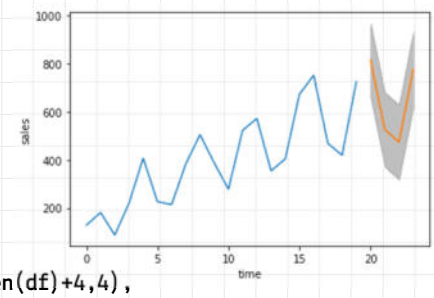
Bottom Right: correlogram (ACF plot) of residual errors. Significant autocorrelations indicate patterns in the data not explained by the model.



Fil Rouge, SARIMA (statsmodels)

Forecast, out-of-sample

```
forewrap = sfit.get_forecast(steps=4)
forecast_ci = forewrap.conf_int()
forecast_val = forewrap.predicted_mean
plt.plot(df.sales)
plt.fill_between(np.linspace(len(df),len(df)+4,4),
                 forecast_ci[:, 0],
                 forecast_ci[:, 1], color='k', alpha=.25)
plt.plot(np.linspace(len(df),len(df)+4,4),forecast_val)
plt.xlabel('time');
plt.ylabel('sales')
plt.show()
```

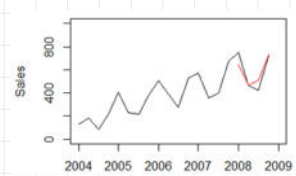


For confidence intervals, see forward

Fil Rouge, forecast errors

Sampling: train 2004-2007 test 2008.

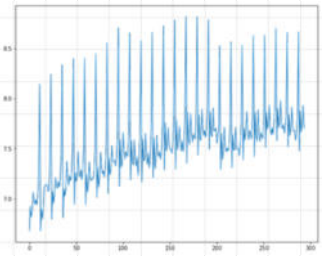
SARIMA model.



ACTUAL	SARIMA					
x_i	f_i	$x_i - f_i$	$\lvert x_i - f_i \rvert$	$(x_i - f_i)^2$		
752	646.03	105.97	105.97	11230.53	BIAS	3.97
468	465.52	2.48	2.48	6.17	MAD	50.26
419	506.92	-87.92	87.92	7729.49	RMSE	68.90
725	729.66	-4.66	4.66	21.68	MAPE	8.73

Example jewelry stores, pmdarima

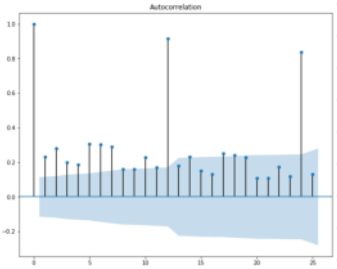
A complete example, with ARIMA



```
import pandas as pd, numpy as np, os
import matplotlib.pyplot as plt
import pmdarima as pm          # pip install pmdarima
from statsmodels.tsa.stattools import acf

# data upload
os.chdir(os.path.dirname(os.path.abspath(__file__)))
df = pd.read_csv('gioiellerie.csv', header=0)
df["period"] = df["year"].map(str) +"-" + df["month"].map(str)
df['period'] = pd.to_datetime(df['period'], format="%Y-%m").dt.to_period('M')
#df = df.set_index('period')
aSales =  df['sales'].to_numpy()     # array of sales data
logdata = np.log(aSales)             # log transform
data = pd.Series(logdata)            # convert to pandas series

plt.rcParams["figure.figsize"] = (10,8) # redefines figure size
plt.plot(data.values);plt.show()     # data plot
```


Example jewelry stores

```
# acf plot, industrial
import statsmodels.api as sm
sm.graphics.tsa.plot_acf(data.values, lags=25)
plt.show()

# train and test set
train = data[:-12]
test  = data[-12:]

# simple reconstruction, not necessary, unused
reconstruct = np.exp(np.r_[train,test])
```

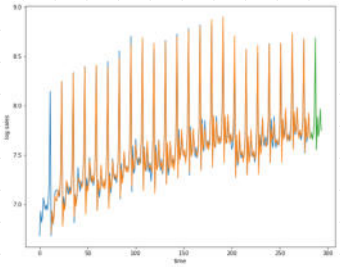


Example jewelry stores

```
# auto arima
model = pm.auto_arima(train.values, start_p=1, start_q=1,
                      test='adf', max_p=3, max_q=3, m=12,
                      start_P=0, seasonal=True,
                      d=None, D=1, trace=True,
                      error_action='ignore',
                      suppress_warnings=True,
                      stepwise=True) # False full grid
print(model.summary())
morder = model.order; print("Sarimax order {0}".format(morder))
mseasorder = model.seasonal_order;
print("Sarimax seasonal order {0}".format(mseasorder))
```

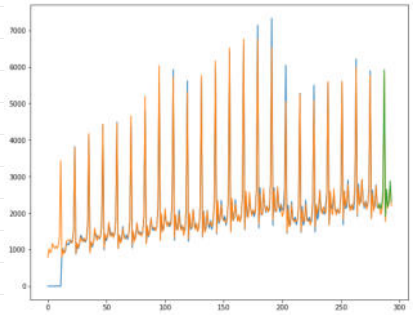

Example jewelry stores

```
# predictions and forecasts
fitted = model.fit(train)
ypred = fitted.predict_in_sample()   # prediction (in-sample)
yfore = fitted.predict(n_periods=12) # forecast   (out-of-sample)

plt.plot(train.values)
plt.plot([None for x in range(12)]+[x for x in ypred[12:]])
plt.plot([None for x in ypred]+[x for x in yfore])
plt.xlabel('time');plt.ylabel('log sales')
plt.show()
```



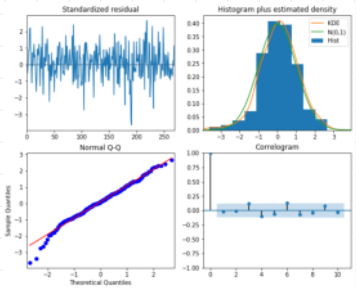
Example jewelry stores

```
# recostruction
yplog = pd.Series(ypred)
expdata = np.exp(yplog)      # unlog
expfore = np.exp(yfore)
plt.plot([None for x in range(12)]+[x for x in expdata[12:]])
plt.plot(df.sales)
plt.plot([None for x in expdata]+[x for x in expfore])
plt.show()
```



Example jewelry stores, statsmodels

```
# ----------------- using statsmodels' SARIMAX, morder derived in auto_arima
from statsmodels.tsa.statespace.sarimax import SARIMAX
sarima_model = SARIMAX(train.values, order=morder, seasonal_order=mseasorder)
sfit = sarima_model.fit()
sfit.plot_diagnostics()
plt.show()

ypred = sfit.predict(start=0,end=len(train))
yfore = sfit.get_forecast(steps=12)
expdata = np.exp(ypred)                    # unlog
expfore = np.exp(yfore.predicted_mean)
plt.plot(expdata)
plt.plot(df.sales)
plt.plot([None for x in expdata]+[x for x in expfore])
plt.show()
```



SARIMAx

Seasonal Autoregressive Integrated Moving-Average with Exogenous Regressors (SARIMAX)

SARIMAX is an extension of SARIMA that also includes the modeling of exogenous variables.

Exogenous variables (also called covariates or predictors or externals) are parallel input sequences that have observations at the same time steps as the original series.

The primary series is referred to as endogenous data to contrast it from the exogenous sequence(s).

The observations for exogenous variables are included in the model directly at each time step and are ***not*** modeled in the same way as the primary endogenous sequence (as an AR, MA, etc. process).

SARIMAx

The SARIMAX method can also be used to model the subsumed models, adding exogenous variables: ARX, MAX, ARMAX, and ARIMAX.

The method is suitable for univariate time series with trend and/or seasonal components and exogenous variables. It predicts a unique time series, using externals.

Seasonal Autoregressive Integrated Moving-Average with Exogenous Regressors (SARIMAX)

SARIMAX is an extension of SARIMA that includes the modeling of exogenous variables.
SARIMAX can also be used to model the subsumed models with exogenous variables, such as ARX, MAX, ARMAX, and ARIMAX.
The method is suitable for time series with trend and/or seasonal components and exogenous variables.

Example with exogenous variables

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
# artificial dataset
data1 = [x + np.random.random() for x in range(1, 100)]
data2 = [x + np.random.random() for x in range(101, 200)]
# fit model
model = SARIMAX(data1, exog=data2, order=(1, 1, 1), seasonal_order=(0, 0, 0, 0))
model_fit = model.fit(disp=False)
# make prediction
exog2 = [200 + np.random.random()]
yhat = model_fit.predict(len(data1), len(data1), exog=[exog2])
print(yhat)
```


Prediction intervals

Complex, here only a hint.

Forecasts are usually wrong, we provide a specific value meaning that the future value will be similar to the suggested one.

Actually, we should provide a value distribution, which can be described by an interval on the expected values.

A hypothesis on the distribution is needed, usually Student's *t*, here normality

The forecast value has means μ, the prediction interval $[\mu-\sigma, \mu+\sigma]$

Prediction intervals

How to estimate the parameter of the distribution when not provided by the API?

One possible way (will be plausible also for neural models):

1. Associate a random variable with each value to forecast (time periods t, t+1, ...).
2. Apply the model on sliding windows on historic data, computing the variance for each lag.
3. Apply the model for the forecast, assuming as prediction interval the forecast value ± the standard deviation computed for the corresponding lag.

Simple Exponential Smoothing

Simple Exponential Smoothing (SES), also *single exponential smoothing*, models the next time step as an exponentially weighted linear function of observations at prior time steps. It requires a single parameter, α, called the smoothing factor or smoothing coefficient.

The equation is simply

$$y_{t+1} = \alpha y_t + (1-\alpha) y_{t-1}$$

That, when projected back in time, corresponds to $y_{t+1} = \alpha y_t + \alpha(1-\alpha) y_{t-1} + \alpha(1-\alpha)^2 y_{t-2} + \ldots$

The parameter controls the rate at which the influence of the observations at prior time steps decay exponentially. Values close to 1 mean that the model pays attention mainly to the most recent observations, whereas values close to 0 mean that more of the history is taken into account.
The method is suitable for univariate time series without trend and seasonal components.

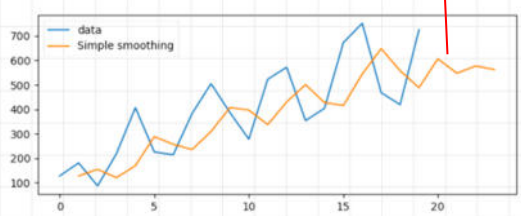
```
def simple_exp_smooth(data, nforecasts=1, alpha=0.4):
    n = len(data)
    f = np.full(n + nforecasts, np.nan)            # Forecast array
    data = np.append(data, [np.nan] * nforecasts)  # forecast placeholders
    f[1] = data[0]                                 # initialization of first forecast
    # predictions
    for t in range(2, n+1):
        f[t] = alpha * data[t - 1] + (1 - alpha) * f[t - 1]
    # forecast
    for t in range(n+1,n+nforecasts):
        f[t] = alpha * f[t - 1] + (1 - alpha) * f[t - 2]

    return pd.DataFrame.from_dict({"Data": data, "Forecast": f, "Error": data - f})
```

Simple Exponential Smoothing

Fil rouge example:

```
sales = pd.read_csv("FilRouge.csv",usecols=["sales"]).T
sales = np.array(sales).flatten()
df = simple_exp_smooth(sales, nforecasts=4,alpha=0.5)
MAE = df["Error"].abs().mean()
print("MAE:", round(MAE, 2))
RMSE = np.sqrt((df["Error"] ** 2).mean())
print("RMSE:", round(RMSE, 2))
df.index.name = "Periods"
plt.figure(figsize=(8, 3))
plt.plot(df[["Data"]],label="data")
plt.plot(df[["Forecast"]],label="Simple smoothing")
plt.legend()
plt.show()
```

Lost the trend
Smoothed
contributions



Holt Winter's Exponential Smoothing (HWES)

Holt Winter's Exponential Smoothing (HWES), also called Triple Exponential Smoothing (there would be a double in-between) models the next time step as an exponentially weighted linear function of observations at prior time steps, taking trends and seasonality into account.

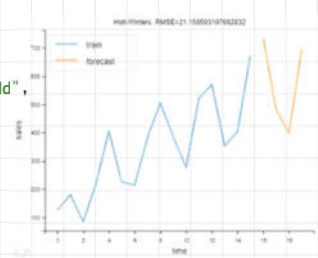
Parameters: α, level smoothing coefficient, β trend coefficient, γ seasonal coefficient.

Trend and seasonality may be modeled as either additive or multiplicative (for a linear or exponential changements, resp.).

The method is suitable for univariate time series with trend and/or seasonal components.

Statsmodels (fil rouge dataseries):

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
# fit model
model = ExponentialSmoothing(train, seasonal_periods=4,trend="add",
    seasonal     ="mul",
    damped_trend = True,
    use_boxcox   = True,
    initialization_method = "estimated")
hwfit = model.fit()
# make forecast
yfore = hwfit.predict(len(train), len(train)+3)
print(yfore)
```



Holt Winter's Exponential Smoothing

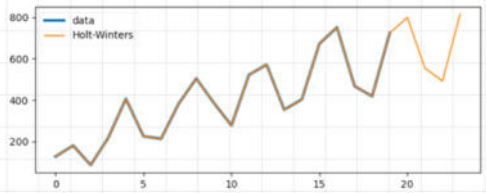
HW complete code, nuts and bolts implementation

```
# x: series, m: seasonality, nfor: num forecast
def holtwinters(x, m, nfor, alpha = 0.5, beta = 0.5, gamma = 0.5):
    Y = x[:]
    initial_values = np.array([0.0, 1.0, 0.0])
    boundaries = [(0, 1), (0, 1), (0, 1)]
    type = 'multiplicative' # could have been additive

    a = [sum(Y[0:m]) / float(m)]
    b = [(sum(Y[m:2 * m]) - sum(Y[0:m])) / m ** 2]
    s = [Y[i] / a[0] for i in range(m)]
    y = [(a[0] + b[0]) * s[0]]
    rmse = 0

    for i in range(len(Y) + nfor):
        if i == len(Y):
            Y.append((a[-1] + b[-1]) * s[-m])

        a.append(alpha * (Y[i] / s[i]) + (1 - alpha) * (a[i] + b[i]))
        b.append(beta * (a[i + 1] - a[i]) + (1 - beta) * b[i])
        s.append(gamma * (Y[i] / (a[i] + b[i])) + (1 - gamma) * s[i])
        y.append((a[i + 1] + b[i + 1]) * s[i + 1])

    rmse = np.sqrt(sum([(m-n)**2 for m, n in zip(Y[:-nfor], y[:-nfor-1])]) / len(Y[:-nfor]))
    return Y, rmse
```


Holt Winter's Exponential Smoothing

Fil rouge example, custom implementation (prev. slide)

```
sales = pd.read_csv("FilRouge.csv",usecols=["sales"]).T
sales = np.array(sales).flatten()
hwforecasts,rmse=holtwinters(sales.tolist(),4,4,alpha=0.6,beta=0.5,gamma=0.3)
print("RMSE HW:", round(rmse, 2))
plt.figure(figsize=(8, 3))
plt.plot(sales,linewidth=3,label="data")
plt.plot(hwforecasts,label="Holt-Winters")
plt.legend()
plt.show()
```



Theta

https://www.sciencedirect.com/science/article/pii/S0169207000000662

The Theta model of Assimakopoulos & Nikolopoulos (2000) is a simple method for forecasting that involves:

- fitting two lines:
  - The Original Time Series (Theta = 1): the raw data without modification, it retains the trend and seasonal patterns present in the data.
  - The Damped Trend Line (Theta = 2): it emphasizes the long-term trend. It is derived by applying a double exponential smoothing (Holt's method with no dampening) to estimate a more stable trend.
- forecasting the lines using a Simple Exponential Smoother,
- combining the forecasts from the two lines to produce the final forecast.

Theta

A seasonality test examines the ACF at the seasonal lag m. If this lag is significantly different from zero then the data is deseasonalized into either a multiplicative or additive method.

The parameters of the model are α and β_0 where β_0 is estimated from the OLS regression

$$x(t) = a_0 + b_0(t-1) + \epsilon_t$$

and α is the SES smoothing parameter in

$$\hat{x}(t) = \alpha x(t) + (1-\alpha)\hat{x}(t-1)$$

The forecasts are then

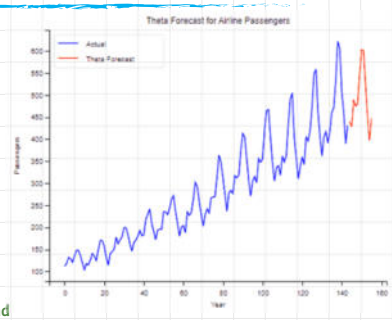
$$\hat{x}(T+h|T) = \frac{\theta - 1}{\theta}\left(h - 1 + \frac{1}{\alpha} + \frac{(1-\alpha)^T}{\alpha}\right) + \hat{x}(T+h|T)$$

Ultimately θ only plays a role in determining how much the trend is damped. If θ is very large, then the forecast of the model is identical to that from an Integrated Moving Average with a drift,

Finally, the forecasts are reseasonalized if needed.

Theta

```
import matplotlib.pyplot as plt
from statsmodels.datasets import get_rdataset
from statsmodels.tsa.forecasting.theta import ThetaModel

# Load the Box & Jenkins airline passenger dataset
airpass = get_rdataset('AirPassengers').data.value.values

# Fit the Theta model
theta_model = ThetaModel(airpass,period=12)
fit = theta_model.fit()
print(fit.summary())
forecast = fit.forecast(steps=12)  # Forecast 12 months ahead

# Plot the original series and forecast
plt.figure(figsize=(10, 5))
plt.plot(airpass, label='Actual', color='blue')
plt.plot(range(len(airpass),len(airpass)+12), forecast, label='Theta Forecast', color='red')
plt.xlabel('Year')
plt.ylabel('Passengers')
plt.title('Theta Forecast for Airline Passengers')
plt.legend()
plt.show()
```



END